

Documentação Técnica

Challenge Água Branca
Gestão de Inovação Corporativa

Aplicativo Android nativo para três perfis (Operador, Gestor e Líder), construído com Kotlin, Jetpack Compose e arquitetura MVVM + Clean Architecture.

FIAP · Sprint 1

Sumário

1. Visão geral
2. Stack e arquitetura
3. Estrutura do projeto
4. Modelo de domínio e persistência
5. Integrações com serviços externos
6. Autenticação e sessão
7. Fluxos por perfil
8. UX e diferenciais
9. Como compilar e rodar
10. Entregáveis e critérios atendidos

1. Visão geral

O **Challenge Águia Branca** é um aplicativo Android nativo de gestão da inovação corporativa. Ele integra **estratégia, pessoas, processos e tecnologia** em um único ambiente, atendendo três níveis organizacionais com experiências e ferramentas específicas para cada um deles.

O perfil **Operador** é o nível operacional – responsável pelo registro e captura de ideias e problemas do dia a dia. O perfil **Gestor** ocupa o nível tático, fazendo a curadoria das ideias e transformando as melhores em projetos. O perfil **Líder** é o nível estratégico, definindo as diretrizes da organização e acompanhando o retorno consolidado das iniciativas através de um dashboard executivo.

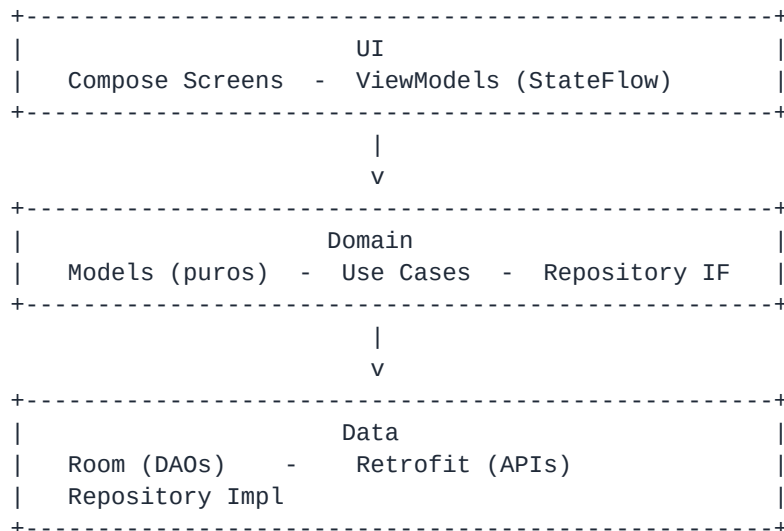
A solução foi pensada para funcionar inteiramente em mobile, com persistência local e sincronização opcional via APIs REST. A arquitetura segue boas práticas de Clean Architecture e MVVM, com clara separação entre as camadas de dados, domínio e interface, permitindo evolução sustentável para a Sprint 2 (substituição dos repositórios locais por implementações remotas sem mudar a UI nem os casos de uso).

2. Stack e arquitetura

A aplicação é **100% Kotlin**, com interface inteiramente declarativa em **Jetpack Compose + Material 3**. A escolha do stack prioriza ferramentas modernas, mantidas pelo Google e amplamente adotadas pelo mercado Android.

Camada	Tecnologia	Justificativa
Linguagem	Kotlin 2.x	Padrão oficial do Android; null-safety, coroutines e sintaxe concisa.
UI	Jetpack Compose + Material 3 (BOM 2026.05.00)	UI declarativa, recomposição eficiente e temas dinâmicos prontos.
Arquitetura	MVVM + Clean Architecture	Separação em data / domain / ui; cada camada com responsabilidade única.
DI	Koin	Injeção leve, sem KAPT; ideal para o ciclo de build do Compose.
Concorrência	Coroutines + Flow	Streams reativos do banco até a UI; suspend functions nos use cases.
Persistência local	Room + DataStore	Banco SQLite com DAO + Flow; DataStore para sugestões do gestor.
Rede	Retrofit + OkHttp + Kotlinx Serialization	Cliente HTTP padrão com (de)serialização type-safe.
Navegação	Compose Navigation	Rotas declarativas com guard por perfil de usuário.
Build	Gradle KTS + Version Catalog	Dependências centralizadas em libs.versions.toml.

Diagrama de camadas (Clean Architecture)



A camada de **UI** não conhece banco nem API: depende somente de ViewModels, que por sua vez orquestram Use Cases do **Domain**. A camada de **Data** implementa as interfaces de repositório definidas no Domain, podendo trocar a fonte (Room ou Retrofit) sem afetar nada acima dela.

3. Estrutura do projeto

A organização de pacotes reflete a divisão em camadas. Cada feature (login, operador, gestor, líder, compartilhado) tem sua própria subárvore dentro de `ui/feature/`, agrupando tela, ViewModel e componentes específicos.

```
br.com.fiap.challengeaguia branca/
+- InnovationApplication.kt      # Inicializacao do Koin
+- di/                          # Modulos Koin (Network, Database, etc.)
+- data/
| +- local/                    # Room: entities, DAOs, database
| +- remote/
| | +- api/                   # Retrofit services
| | +- dto/                   # Respostas das APIs
| | +- mapper/                # DTO -> Domain
| +- repository/               # Implementacoes dos repositorios
+- domain/
| +- model/                    # Modelos puros (Idea, Project, etc.)
| +- repository/               # Interfaces (contratos)
| +- usecase/                  # Casos de uso por feature
+- ui/
| +- theme/                    # Material Theme + cores por perfil
| +- navigation/               # NavGraph e rotas
| +- components/               # Composables reutilizaveis
| +- feature/
| | +- auth/                   # Login
| | +- operator/               # Home, ideias, perfil, estrategias
| | +- manager/                # Dashboard, curadoria, projetos, etc.
| | +- leader/                 # ROI, diretrizes, acompanhamento
| | +- shared/                 # Tela de Colaboradores (chat)
```

4. Modelo de domínio e persistência

O modelo de domínio contém as entidades de negócio principais. Todas são **data classes Kotlin imutáveis**, sem referência a frameworks (puro Kotlin). A persistência é feita em Room através de entidades espelho com mappers explícitos entre DTOs, entities e modelos de domínio.

Tabela (Room)	Modelo (Domain)	Conteúdo
ideas	Idea	Ideia/problema cadastrado pelo operador (título, descrição, categoria, status).
projects	Project	Projeto criado pelo gestor a partir de uma ideia aprovada (investimento, lucro, prazo, status).
strategic_guidelines	StrategicGuideline	Diretriz estratégica criada/editada pelo líder.
user_session	UserSession	Sessão local do usuário autenticado (linha única id=1).

Toda leitura do banco é exposta como **Flow**, permitindo que a UI reaja automaticamente a qualquer mudança sem precisar de polling. Por exemplo, ao o gestor aprovar uma ideia, a tela do operador atualiza o status na lista *Minhas Ideias* sem nenhuma ação adicional.

5. Integrações com serviços externos

O desafio exige consumo efetivo de pelo menos uma API REST. A solução implementa duas, ambas funcionais e não apenas demonstrativas:

Endpoint	Uso	Onde aparece
GET fakerapi.it/api/v1/users	Seed inicial de usuários para os três perfis (nome, e-mail, avatar).	Login – via AuthenticateUserUseCase.
GET api.adviceflip.com/advice	Frase do dia exibida como insight motivacional ao operador.	Home do operador – card Insight do Dia.

As chamadas são feitas com **Retrofit + OkHttp** e desserializadas com **Kotlinx Serialization**. Erros de rede são tratados no use case e transformados em estados explícitos (*loading / empty / error / success*) que a UI consome para apresentar feedbacks adequados ao usuário.

6. Autenticação e sessão

A autenticação é baseada em uma lista de contas autorizadas com mapeamento fixo entre e-mail e perfil. Ao validar as credenciais, o app busca na FakerAPI um usuário do mesmo perfil para enriquecer com nome e avatar, e então persiste a sessão no Room (linha única em *user_session*).

Perfil	E-mail	Senha
Operador	operador@innovatecorp.com	oper123
Gestor	gestor@innovatecorp.com	gest123
Líder	lideranca@innovatecorp.com	lider123

Na próxima abertura do app, se houver sessão persistida, o usuário é redirecionado automaticamente para a home do seu perfil. O logout limpa a sessão e retorna ao login. A navegação usa **NavGraph guardado por perfil** – cada perfil tem acesso somente às rotas pertinentes ao seu nível.

7. Fluxos por perfil

7.1 Operador

A home do operador apresenta um banner de **gamificação** com nome, nível e pontos calculados a partir da quantidade de ideias enviadas e aprovadas. Logo abaixo, KPIs pessoais (ideias enviadas / aprovadas), o **Insight do Dia** (carregado da AdviceSlip), uma diretriz estratégica em destaque e ações rápidas (Nova Ideia, Reportar Problema, Colaboradores, Ver todas).

A aba **Ideias** oferece formulário simples (título, descrição, categoria) e a listagem das próprias ideias com status visual (Pendente, Aprovada, Reprovada, Priorizada). A aba **Estratégias** mostra as diretrizes em modo somente leitura. A aba **Perfil** permite editar nome, e-mail, avatar (galeria ou cor) e exibe o resumo da participação.

O card **Colaboradores** abre um chat 1:1 com os colegas diretos (filtrado pela relação do perfil), mantendo o menu inferior sempre acessível para o usuário não perder o contexto da navegação principal.

7.2 Gestor

O dashboard tático apresenta KPIs agregados da equipe (total de ideias, pendentes, projetos em andamento) e um **gráfico de barras** com a distribuição de ideias por status. A partir dele, o gestor acessa diretamente a curadoria das pendências.

Na aba **Curadoria**, cada ideia recebe três ações possíveis: *Aprovar*, *Reprovar* ou *Priorizar*. A mudança reflete imediatamente no banco e em todas as outras telas que observam aquela ideia. A aba **Projetos** permite criar iniciativas a partir de ideias aprovadas/priorizadas, registrando investimento, prazo, status de execução e o retorno financeiro obtido.

O gestor também pode **enviar sugestões** para a liderança (uma forma de comunicação estruturada com o nível estratégico), consultar as diretrizes em modo leitura e acessar o chat com a equipe direta.

7.3 Líder

A home do líder é um **dashboard executivo** que consolida ROI, investimento acumulado, lucro consolidado, prazo e ganhos de produtividade dos projetos em execução. Esses números são derivados em tempo real dos dados que o gestor registra na aba Projetos, evidenciando o ciclo completo: operação → tático → estratégico.

A aba **Diretrizes** é o único ponto de CRUD completo dessa entidade: somente o líder pode criar, editar ou excluir orientações estratégicas. A aba **Acompanhamento** traz a visão completa dos projetos com investimento, lucro, ROI individual, prazo e status. A home também disponibiliza um **ranking** mensal e geral da equipe, além do canal de recebimento das sugestões enviadas pelos gestores.

8. UX e diferenciais

Além das funcionalidades requisitadas pelo desafio, a solução incorpora vários elementos de UX que elevam a experiência:

- **Tema dinâmico claro/escuro** com alternância em tempo real pelo ícone do login – todas as cores têm versão adaptada para os dois modos.
- **Identidade visual por perfil**: cada nível tem uma paleta própria, facilitando o reconhecimento visual imediato em qual ambiente o usuário está.
- **Animações premium** de transição entre abas e entrada de listas, implementadas com a API de animações do Compose.
- **Tratamento correto de insets do sistema** – barra de status, gesture pill e teclado – usando `WindowInsets.safeDrawing` e `imePadding` onde apropriado.
- **Notificações in-app** com badge de não lidos: cada perfil recebe alertas contextuais (ex.: gestor vê quando há ideias novas pendentes).
- **Componentes reutilizáveis** (`InnovateCard`, `InnovateScreenBackground`, `QuickActionCard`, `KpiStatCard`) que mantêm consistência visual e reduzem duplicação.

9. Como compilar e rodar

Para executar o projeto a partir do código-fonte:

1. Abra o projeto no Android Studio (Giraffe ou superior).
2. Aguarde o Sync do Gradle baixar as dependências.
3. Selecione um dispositivo (emulador ou físico) com API 28 ou superior.
4. Execute Run / Debug.

Para gerar o APK debug pela linha de comando (no Windows PowerShell):

```
.\gradlew.bat assembleDebug
```

APK gerado em: `app\build\outputs\apk\debug\app-debug.apk`

10. Entregáveis e critérios atendidos

Entregável	Arquivo	Status
APK Android (debug)	aguiabranca-debug.apk	Incluído
Código-fonte completo	codigo-fonte-completo.zip	Incluído
Documentação técnica	documentacao/documentacao-tecnica.pdf	Este documento
Versão editável da doc.	documentacao/documentacao-tecnica.pptx	Incluído
Roteiro do vídeo demo	video/roteiro-video-demonstrativo.md	Incluído (MP4 a gravar)

Critérios do desafio atendidos:

- Três perfis distintos com fluxos dedicados (Operador, Gestor, Líder).
- Autenticação com gerenciamento seguro de sessão (Room).
- CRUD de diretrizes restrito ao Líder; consulta disponível para os demais.
- Cadastro/consulta de ideias pelo Operador; curadoria com aprovar/reprovar/priorizar pelo Gestor.
- CRUD completo de projetos pelo Gestor com atualização de progresso e resultados.
- Acompanhamento dos projetos pelo Líder com prazo, status, investimento e retorno.
- Dashboard executivo do Líder com ROI, lucro, investimento e produtividade.
- Conectividade externa funcional: duas APIs REST consumidas e Room como banco local.